MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana

Department of Computer Science and Engineering

BACKEND ENGINEERING AND SYSTEMS CHALLENGING (APEX)

PROJECT DOCUMENTATION

Date: 10-10-2026 | Venue: 2rd Floor, A & B Blocks

Field	Details
Project Title	Emergency resource sharing System
Team Name / Team No.	9
Year / Semester / Section	II-Year B.Tech / 2nd Sem /1
Algorithm Paradigm Used	Searching algorithm, sorting, CRUD Operations, Filtering Algorithm, Location Based Search
Programming Language / Tools	Frontend: React.js, runtime: node.js, backend MongoDB Atlas
Faculty Mentor	Alekhya Pillarishetty

Team Members

S.No	Name of the Student	Roll No.	Role
1	S. Sai Rithvika	25MVCSMR0527	Team Lead
2	B. Keerthi Sri	25MVCSMR0476	
3	K. Keerthi Reddy	25MVCSMR0496	
4	A.Deekshitha	25MVCSMR0459	

1. Problem Statement

The Emergency Resource Sharing System using MERN Stack aims to solve the problem of finding and sharing essential resources during emergencies such as floods, accidents, and natural disasters.

People often face difficulties locating food, water, medicines, clothes, and shelter when they need them urgently. This system provides a centralized web platform to manage resource details, including quantity, location, owner, and availability, helping people find and share resources efficiently during emergencies.

2. Objectives

- 1. Emergency Resource Sharing: To connect people who need emergency resources with people who can provide help.
- 2. Quick Emergency Requests: To allow users to submit requests for food, water, medicines, blood, shelter, and rescue services.
- 3. Real-Time Coordination: To help requesters and service providers coordinate emergency assistance through a centralized platform.
- 4. User Authentication: To provide secure registration and login for requesters and resource providers.
- 5. Request Management: To track emergency requests based on their status, such as Pending, Accepted, and Completed.
- 6. Priority-Based Assistance: To identify high-priority and critical emergency requests for better response management.
- 7. Resource Utilization: To improve the effective distribution of available resources and reduce delays in providing assistance.
- 8. User-Friendly Interface: To develop an accessible and easy-to-use platform using modern web technologies.
- 9. Database Management: To securely store and manage user details and emergency request information.
- 10. Scalable Solution: To build a platform that can be enhanced in the future with location-based services, notifications, and emergency response tracking.
- Main Objective: To develop a centralized web platform that connects people in need with resource providers and helps coordinate emergency assistance efficiently.
- To support effective resource management during natural disasters and other emergencies.

3. Proposed Approach

Chosen Paradigm: Searching and Filtering

- Searching: Linear Search can be used to find resources by name, type, or location.
- Filtering: Filters resources based on availability, category, and location.
- Sorting: Resources can be sorted by quantity or other relevant criteria to make results easier to review.

Why It Suits This Problem:

These techniques help users find essential resources such as food, water, medicines, and shelter quickly. They are simple to implement and suitable for a web application that manages resource information stored in MongoDB.

Alternatives Considered:

- Binary Search: Can search sorted data efficiently but requires the data to be sorted by the search field.
- Hashing: Can provide fast lookups using exact keys, such as a resource ID, but is less suitable for flexible searches across multiple fields.
- Graph Algorithms: Could help find the shortest route to a resource when map and road-network data are available.

Conclusion: Searching and filtering are suitable starting points for this project because they support practical resource discovery. The final choice should match the algorithms implemented in the actual code.

4. Algorithm / Pseudocode

START

DISPLAY Login and Registration Page

IF user has valid saved session, THEN
DISPLAY Welcome Page
ELSE
DISPLAY Login Page
END IF

IF user registers THEN
GET name, email, password, and role
VALIDATE user details
SAVE user details in database
DISPLAY registration success message
END IF

IF user logs in THEN
VERIFY email and password

IF credentials are valid THEN

GENERATE authentication token

SAVE token and user details

DISPLAY Welcome Page

ELSE

DISPLAY login error

END IF

END IF

IF user clicks Explore Dashboard, THEN

DISPLAY Main Dashboard

IF user role is Requester, THEN

DISPLAY Requester Options

ELSE IF user role is Provider THEN

DISPLAY Provider Options

END IF

END IF

END

6. Implementation

1. User Authentication Module

- Allows users to register and log in securely.
- Supports two user roles: Requester and Provider.
- Uses password hashing and JWT-based authentication.
- Provides logout functionality.

2. Emergency Request Management Module

- Allows requesters to create emergency requests.
- Collects details such as title, category, description, quantity, location, phone number, and priority.
- Stores request information in the database.
- Assigns a default status of Pending to new requests.

3. Resource Provider Module

- Allows providers to view emergency requests.
- Enables providers to accept available requests.

- Allows assigned providers to mark accepted requests as Completed.

4. Dashboard Module

- Displays emergency request statistics, including total, pending, accepted, and completed requests.
- Provides navigation to emergency request management.
- Allows users to refresh request information.

5. Request Tracking Module

- Allows requesters to view their own requests and their current status.
- Helps providers monitor requests they can respond to.
- Supports status updates throughout the request lifecycle.

6. API and Database Module

- Provides REST API endpoints for authentication and emergency request operations.
- Validates incoming requests and manages communication between the frontend and database.
- Stores user and emergency request records in MongoDB.

2. Data Structures Used

User Object: Stores user ID, name, email, hashed password, and role.

Emergency Request Object: Stores request ID, requester ID, title, category, description, quantity, location, phone number, priority, status, and timestamps. An assigned provider reference is used where implemented.

Arrays: Used in React to store and display lists of emergency requests.

Objects: Used to represent users, emergency requests, form values, API responses, and dashboard statistics.

Enumerated Values: Roles include Requester and Provider. Request categories include Food, Water, Medicine, Blood, Shelter, Rescue, and Other. Request priorities include Normal, High, and Critical. Status values include Pending, Accepted, Completed, and Cancelled.

Database Collections: MongoDB stores user records and emergency request documents. Mongoose schemas define their fields, validation rules, and relationships.

3. Tools and Platforms

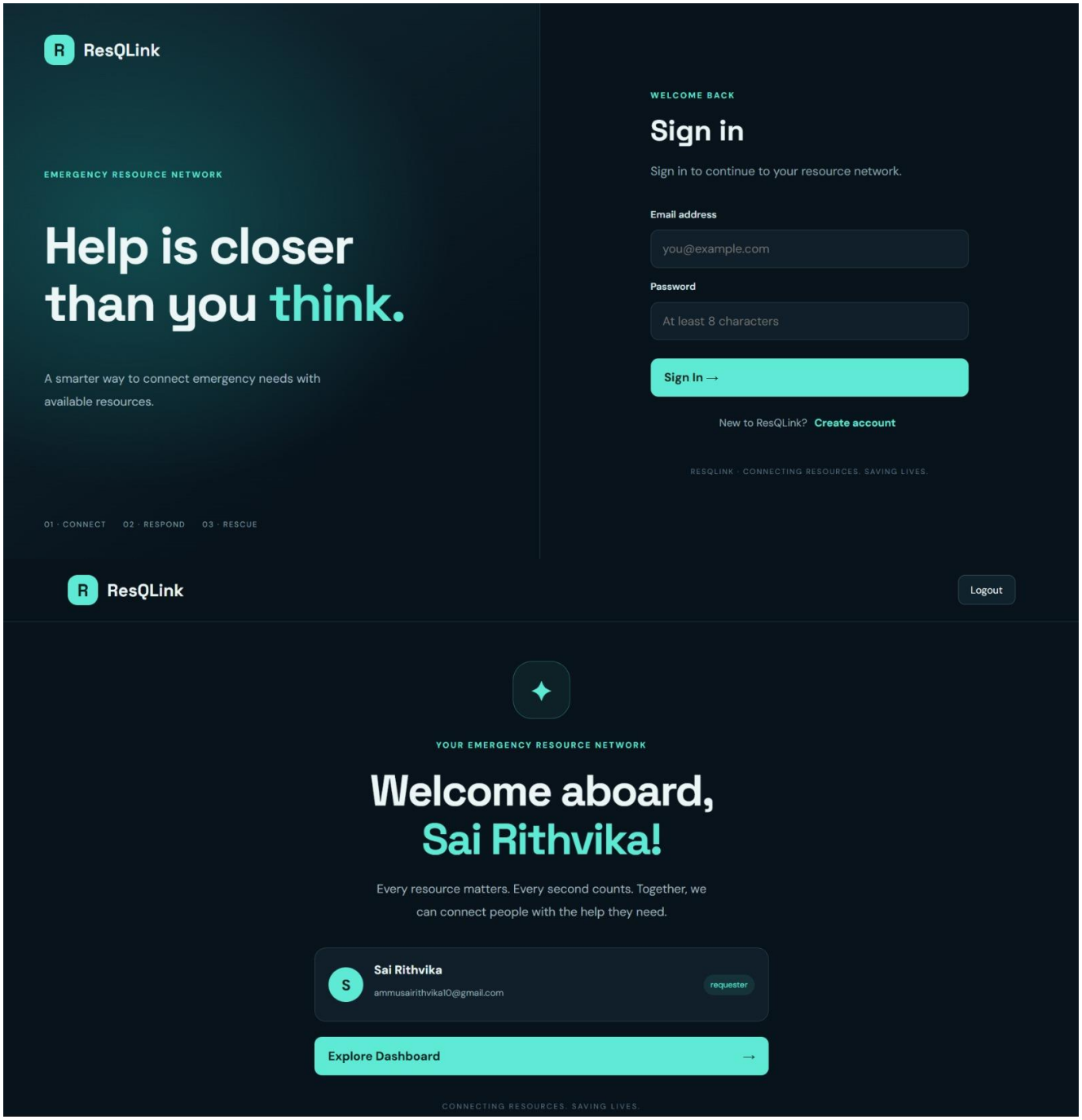
Tool / Technology	Purpose
React.js	Builds the frontend user interface

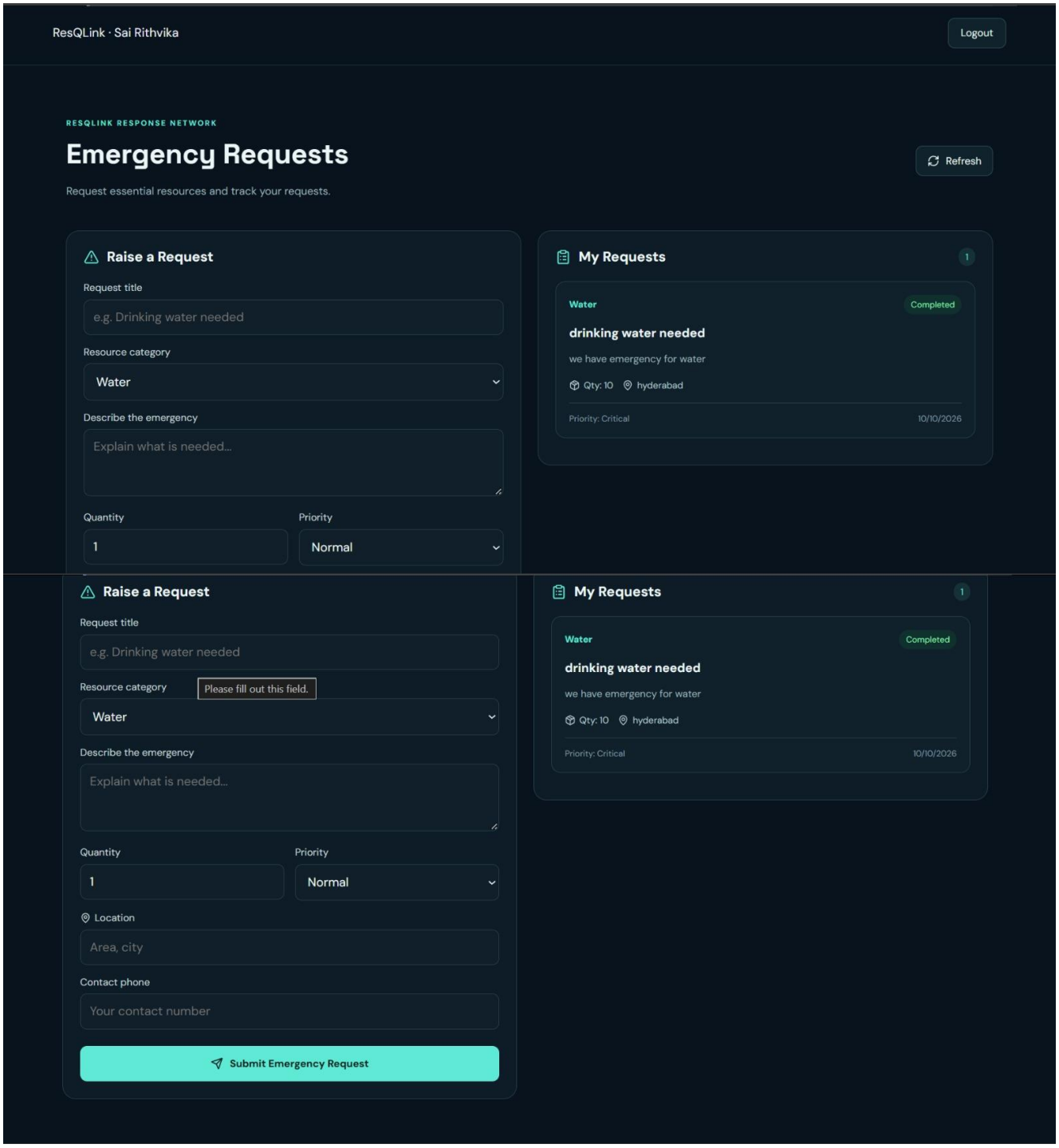
JavaScript	Implements frontend and backend application logic
Node.js	Runs the backend JavaScript application
Express.js	Creates REST APIs and handles HTTP requests
MongoDB Atlas	Stores user and emergency request data
Mongoose	Defines schemas and interacts with MongoDB
JWT	Authenticates users and protects API access
bcryptjs	Hashes passwords securely
HTML and CSS	Structures and styles the interface
Framer Motion	Adds interface animations
Lucida React	Provides interface icons
Vite	Runs and builds the React frontend
Visual Studio Code	Development environment
Git and GitHub	Version control and source code hosting
Postman / PowerShell	Tests API endpoints

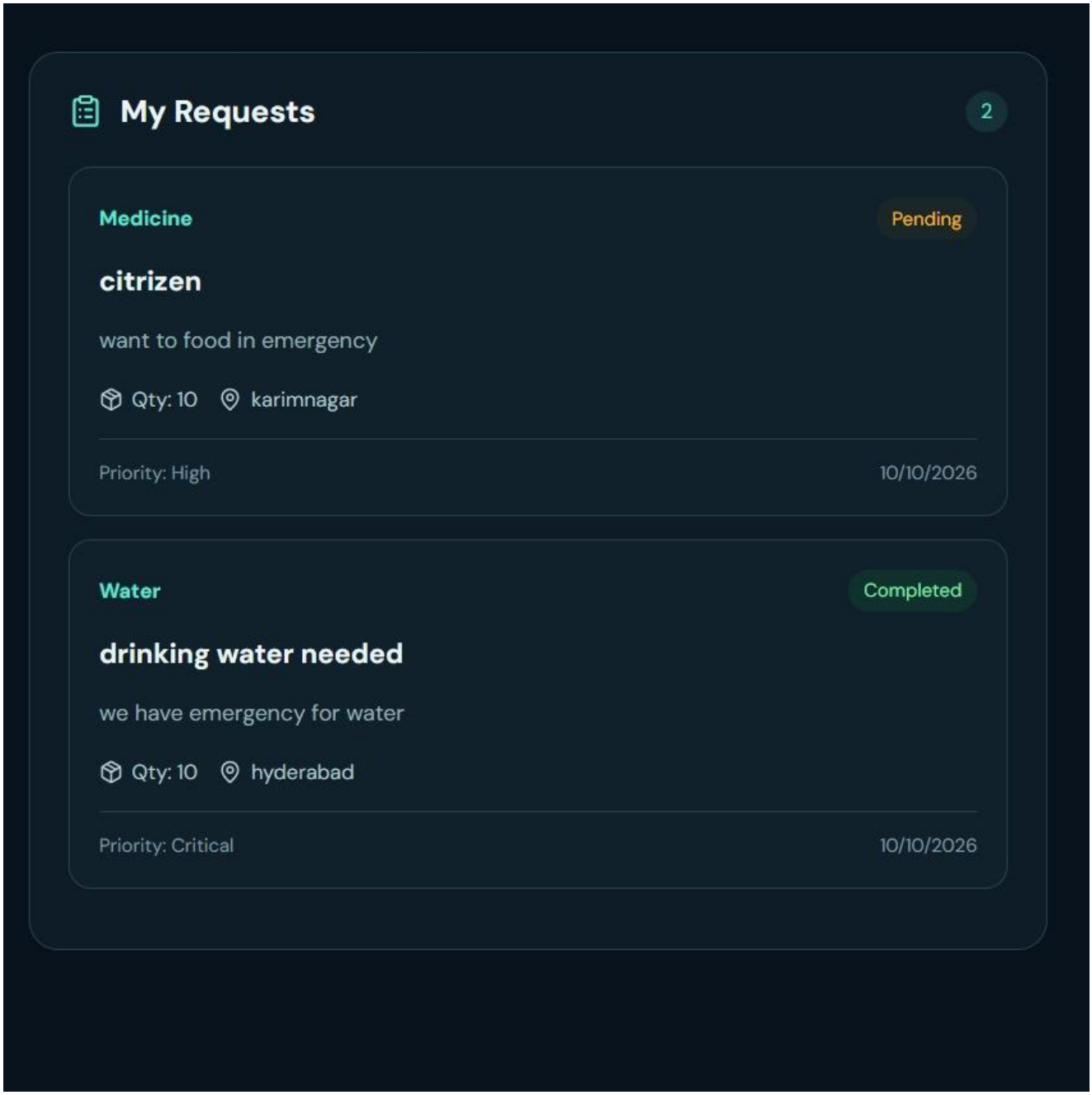
4. Project Workflow

1. A user registers or logs in.
2. The backend verifies credentials and issues an authentication token.
3. The application identifies the user's role.
4. A requester creates an emergency request, or a provider views available requests.
5. Providers accept requests and update their status after completing the task.
6. The dashboard retrieves request data and displays relevant statistics.
7. MongoDB stores the application data for later retrieval.

7. Results and Test Cases


ResQLink
EMERGENCY RESOURCE NETWORK
Help is closer than you think.
A smarter way to connect emergency needs with available resources.
01 · CONNECT
02 · RESPOND
03 · RESCUE
WELCOME BACK
Sign in
Sign in to continue to your resource network.
Email address
you@example.com
Password
At least 8 characters
Sign In →
New to ResQLink? Create account
RESQLINK · CONNECTING RESOURCES, SAVING LIVES.
ResQLink
Logout
YOUR EMERGENCY RESOURCE NETWORK
Welcome aboard, Sai Rithvika!
Every resource matters. Every second counts. Together, we can connect people with the help they need.
S
Sai Rithvika
ammusairithvika10@gmail.com
requester
Explore Dashboard →
CONNECTING RESOURCES, SAVING LIVES.


ResQLink · Sai Rithvika
Logout
RESQLINK RESPONSE NETWORK
Emergency Requests
Request essential resources and track your requests.
Refresh
Raise a Request
Request title
e.g. Drinking water needed
Resource category
Water
Describe the emergency
Explain what is needed...
Quantity
1
Priority
Normal
My Requests
1
Water
Completed
drinking water needed
we have emergency for water
Qty: 10
hyderabad
Priority: Critical
10/10/2026
Raise a Request
Request title
e.g. Drinking water needed
Resource category
Please fill out this field.
Water
Describe the emergency
Explain what is needed...
Quantity
1
Priority
Normal
Location
Area, city
Contact phone
Your contact number
Submit Emergency Request
My Requests
1
Water
Completed
drinking water needed
we have emergency for water
Qty: 10
hyderabad
Priority: Critical
10/10/2026



8. Conclusion and Future Scope

ResQLink is a MERN stack-based web application designed to simplify emergency resource sharing and coordination. The platform connects people who need emergency assistance with resource providers through a centralized system.

The application supports user registration and login, emergency request creation, request management, and status tracking. It uses React for the user interface, Node.js and Express.js for backend services, and MongoDB for storing user and emergency request information.

ResQLink aims to improve the organization of emergency assistance, reduce the difficulties associated with manual coordination, and make essential resources such as food, water, medicines, blood, shelter, and rescue services easier to request and manage.

The project provides a foundation for a scalable emergency coordination platform. Further development and real-world testing can help improve its reliability, usability, and effectiveness during emergency situations.

9. References

[1] React, "React Documentation." [Online]. Available: https://react.dev/. [Accessed: Oct. 10, 2026].

[2] OpenJS Foundation, "Node.js Documentation." [Online]. Available: https://nodejs.org/docs/. [Accessed: Oct. 10, 2026].

[3] Express.js, "Express—Node.js Web Application Framework." [Online]. Available: https://expressjs.com/. [Accessed: Oct. 10, 2026].

[4] MongoDB, Inc., "MongoDB Manual." [Online]. Available: https://www.mongodb.com/docs/manual/. [Accessed: Oct. 10, 2026].

[5] Mongoose, "Mongoose Documentation." [Online]. Available: https://mongoosejs.com/docs/. [Accessed: Oct. 10, 2026].

[6] OWASP Foundation, "JSON Web Token Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_Cheat_Sheet.html. [Accessed: Oct. 10, 2026].

[7] OWASP Foundation, "NodeJS Security Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/NodeJS_Security_Cheat_Sheet.html. [Accessed: Oct. 10, 2026].

[8] Vite, "Vite Documentation." [Online]. Available: https://vite.dev/guide/. [Accessed: Oct. 10, 2026].